

API Development — The Complete Guide

REST • RESTful Standards • Swagger/OpenAPI • GraphQL • gRPC • SOAP

For Java Full Stack Developers (Spring Boot + React)

Why APIs, Design, Security, Resilience, Real Organization Implementation, Interview Q&A

A practical, simple-language, end-to-end reference for interview preparation and real-world implementation

Table of Contents

1. Why APIs Are Needed — Life Before APIs
2. What Is an API? Simple Explanation with Real-World Analogy
3. SOAP — What It Is and Why REST Replaced It (Mostly)
4. Types of APIs — REST, SOAP, GraphQL, gRPC, WebSocket Overview
5. REST Principles Explained Simply
6. Resource-Based URL Design
7. HTTP Methods (GET, POST, PUT, PATCH, DELETE)
8. HTTP Status Codes — Full Real Reference
9. Request & Response Design and the DTO Pattern
10. API Versioning Strategies
11. Pagination, Sorting & Filtering
12. HATEOAS — Basic Understanding
13. Idempotency in REST APIs
14. Error Handling & Global Exception Handling (Spring Boot)
15. Request Validation (Bean Validation)
16. OpenAPI / Swagger Deep Dive
17. Rate Limiting for APIs
18. API Security — JWT Authentication
19. API Security — OAuth2
20. CORS Explained
21. API Gateway Integration
22. Logging & Auditing for APIs
23. API Documentation Best Practices
24. REST API Standards — Naming Conventions & Consistent Response Structure
25. Correlation ID & Trace ID Propagation
26. Response Compression
27. API Backward Compatibility
28. Retry Strategy, Timeout Handling & Circuit Breaker for APIs
29. Caching Strategy for APIs
30. Monitoring & Observability for APIs
31. GraphQL — Architecture, Schema, Query, Mutation, Subscription
32. GraphQL Resolvers, Data Fetching, the N+1 Problem & DataLoader
33. GraphQL Security, Query Complexity, Pagination & Federation
34. GraphQL vs REST — Full Comparison
35. gRPC — Protocol Buffers & Service Definitions
36. gRPC Call Types — Unary, Server Streaming, Client Streaming, Bidirectional
37. gRPC Deep Dive — Serialization, Stubs, Deadlines, Auth, TLS, Interceptors
38. gRPC vs REST — When to Use Which
39. Real-Time API Design — Contract-First & Consumer-Driven Contracts
40. Resilience Patterns for APIs (Recap: Circuit Breaker, Retry, Bulkhead, Timeout, Fallback)
41. Event-Driven & Asynchronous API Communication

- 42. SLA & SLO Considerations for APIs
- 43. Postman and Other API Development Tools
- 44. How We Implement APIs in Our Organization (End-to-End Spring Boot)
- 45. APIs and Databases — DTO vs Entity, Avoiding the N+1 Query Problem
- 46. How the Frontend (React) Consumes Our APIs
- 47. Types of Applications That Consume APIs (Web, Mobile, Third-Party, IoT)
- 48. Third-Party API Integration & External API Calls
- 49. Common API Issues, Problems & Solutions
- 50. Interview Questions & Answers (Basic → Advanced → Scenario Based)
- 51. Final Summary & Cheat Sheet

1. Why APIs Are Needed — Life Before APIs

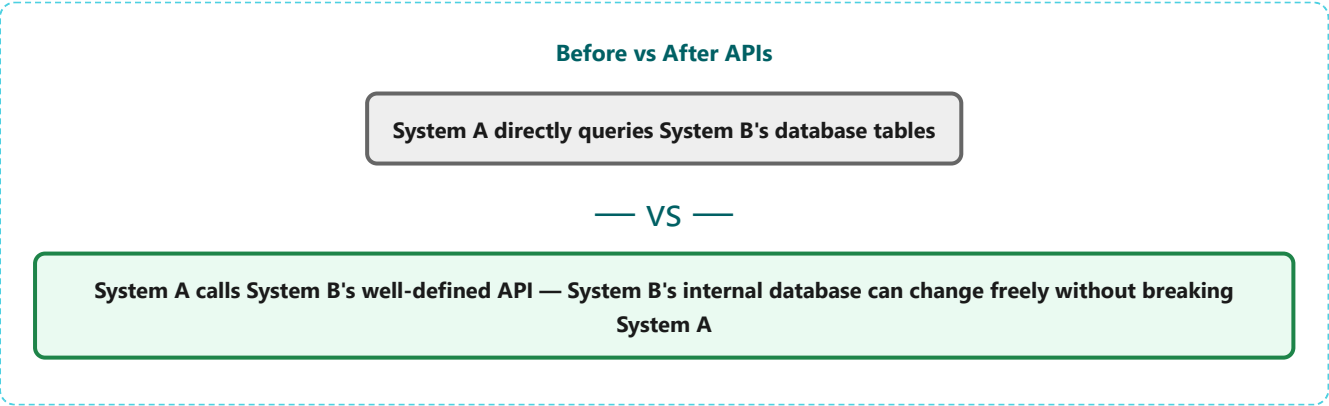
1.1 Life Before APIs

Before standardized APIs, applications that needed to share data used tightly coupled, fragile approaches — direct database access from multiple applications, file-based exports (CSV/FTP transfers run nightly), or proprietary binary protocols understood only by the two specific systems involved. Every integration was custom, one-off, and brittle.

1.2 Real Problems This Caused

Old Approach	Real Problem
Direct database access between systems	Any schema change in one system silently broke every other system reading that table directly
Nightly file exports (CSV/FTP)	Data was always stale by up to 24 hours; no real-time updates possible
Screen-scraping (parsing another app's HTML/UI)	Extremely fragile — any UI change broke the integration instantly
Custom point-to-point protocols	Every new integration required custom code understood by only two teams, not reusable

1.3 What APIs Actually Solve



- **Decoupling** — the consumer only depends on a stable contract (the API), not internal implementation details.
- **Real-time access** — data is fetched live, not from yesterday's export file.
- **Reusability** — one well-designed API can serve a web app, a mobile app, and third-party partners simultaneously.
- **Security boundary** — the API controls exactly what's exposed, instead of granting broad direct database access.
- **Language/platform independence** — a Java backend can serve a React frontend, an iOS app, and a partner's Python script, all through the same HTTP-based API.

Interview one-liner: "APIs exist to let independently built, independently deployed systems communicate through a stable, well-defined contract — without needing to know or depend on each other's internal implementation."

2. What Is an API? Simple Explanation with Real-World Analogy

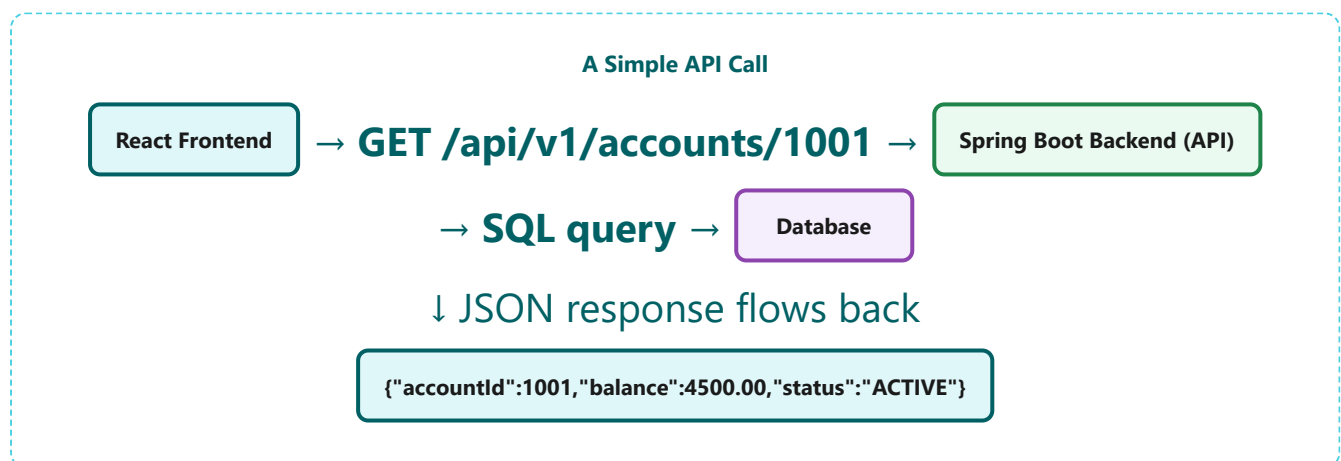
2.1 The Restaurant Analogy

Think of a restaurant: you (the client) don't walk into the kitchen (the server/database) to cook your own food. You give your order to a waiter (the API) using a menu (the API contract) — the waiter takes your request to the kitchen, and brings back exactly what you asked for, in a standard format, without you ever needing to know how the kitchen actually works.

2.2 Formal Definition

An API (Application Programming Interface) is a defined set of rules and endpoints that allows one piece of software to request services or data from another, specifying exactly what requests look like and what responses to expect — without exposing internal implementation details.

2.3 A Real API Call, End to End



3. SOAP — What It Is and Why REST Replaced It (Mostly)

3.1 What Is SOAP?

SOAP (Simple Object Access Protocol) is an older, strict, XML-based messaging protocol for web services, popular in the early-to-mid 2000s enterprise world — every request and response is an XML document following a formal envelope structure, and every service publishes a WSDL (Web Services Description Language) file formally describing every available operation.

3.2 A Real SOAP Request/Response

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <GetAccountBalance xmlns="http://bank.com/services">
      <accountId>1001</accountId>
    </GetAccountBalance>
  </soap:Body>
</soap:Envelope>

<!-- Response -->
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <GetAccountBalanceResponse>
      <balance>4500.00</balance>
      <status>ACTIVE</status>
    </GetAccountBalanceResponse>
  </soap:Body>
</soap:Envelope>
```

3.3 SOAP vs REST — Why the Industry Moved to REST

Aspect	SOAP	REST
Format	XML only, verbose envelope structure required	Usually JSON (lightweight), but can be XML too
Protocol	Can run over HTTP, SMTP, TCP — protocol-agnostic	Built specifically around HTTP and its semantics
Contract	Strict WSDL contract required	Flexible — OpenAPI/Swagger optional but common
Built-in standards	WS-Security, WS-AtomicTransaction (heavy but formal enterprise standards)	No built-in standards — relies on conventions (JWT, OAuth2) layered on top
Performance	Heavier due to XML verbosity and parsing overhead	Lighter, faster, more human-readable
Where it's still used today	Legacy banking/insurance systems, some government/enterprise integrations requiring formal contracts	Virtually all modern web/mobile/microservices APIs

Real scenario: "Our Loans Service still integrates with one legacy credit bureau system that only exposes a SOAP endpoint from the early 2000s — we wrap it in a small internal adapter service that exposes a clean REST API to the rest of our

microservices, so the SOAP complexity is isolated in one place instead of spreading across the codebase."

4. Types of APIs — REST, SOAP, GraphQL, gRPC, WebSocket Overview

API Type	Style	Best Fit
REST	Resource-based, HTTP verbs, usually JSON	The default choice for most public/internal web and mobile APIs
SOAP	XML envelope, WSDL contract, protocol-agnostic	Legacy enterprise integrations, strict formal contracts
GraphQL	Single endpoint, client specifies exactly what data it needs	Complex UIs needing flexible, aggregated data fetching (Section 30)
gRPC	Binary Protobuf over HTTP/2, strongly-typed contracts	High-throughput internal service-to-service calls (Section 34)
WebSocket	Persistent, full-duplex connection	Real-time bidirectional needs — chat, live notifications, live dashboards

Classifying by Communication Style



This guide's main focus (as requested) is REST/RESTful API development and Swagger/OpenAPI documentation, with dedicated deep dives into GraphQL and gRPC as important alternatives you should understand for interviews and the right real-world use case.

5. REST Principles Explained Simply

5.1 What REST Actually Means

REST (Representational State Transfer) is an architectural style (not a strict protocol like SOAP) built around the idea that everything is a **resource** (a noun, like "account" or "order"), identified by a URL, manipulated using standard HTTP verbs, with each request being self-contained (stateless).

5.2 The Six REST Constraints

Constraint	Meaning
Client-Server	Frontend and backend are separate, independently evolvable concerns
Statelessness	Every request contains all information needed to process it — the server holds no client session state between requests
Cacheability	Responses explicitly indicate whether they can be cached, to improve performance
Uniform Interface	Consistent resource naming, standard HTTP verbs, standard status codes across the whole API
Layered System	Client doesn't need to know if it's talking directly to the server or through a gateway/proxy/load balancer
Code on Demand (optional)	Rarely used — server can optionally send executable code (e.g., JavaScript) to the client

5.3 REST vs RESTful — Is There a Difference?

"REST" refers to the architectural style/constraints themselves; "RESTful" is the common term for an API that actually follows those constraints properly. In practice, most real-world "REST APIs" are RESTful to varying degrees — few implement HATEOAS (Section 12) fully, for example, but still call themselves RESTful.

6. Resource-Based URL Design

6.1 Think in Nouns, Not Verbs

A very common mistake: designing URLs like actions/RPC calls (`/getAccount` , `/createOrder` , `/deleteCard`) instead of resources. REST URLs should represent **things** (nouns), and the HTTP method itself represents the action.

Bad (RPC-style)	Good (Resource-based REST)
GET <code>/getAccountById?id=1001</code>	GET <code>/accounts/1001</code>
POST <code>/createNewAccount</code>	POST <code>/accounts</code>
POST <code>/deleteAccount?id=1001</code>	DELETE <code>/accounts/1001</code>
GET <code>/getAllCardsForAccount?accountId=1001</code>	GET <code>/accounts/1001/cards</code>

6.2 Real URL Design for the Banking Example

```
GET    /api/v1/accounts           # list accounts
GET    /api/v1/accounts/1001    # get one account
POST   /api/v1/accounts         # create a new account
PUT    /api/v1/accounts/1001    # full update
PATCH /api/v1/accounts/1001    # partial update
DELETE /api/v1/accounts/1001    # close/delete account
GET    /api/v1/accounts/1001/transactions # nested resource – transactions belonging to this account
GET    /api/v1/accounts/1001/cards/8890  # deeply nested resource
```

6.3 URL Design Rules of Thumb

- Use plural nouns (`/accounts` , not `/account`) — consistent across the whole API.
- Use lowercase, hyphen-separated words for multi-word resources (`/loan-applications` , not `/loanApplications`).
- Nest resources only 1-2 levels deep — beyond that, consider flattening with query parameters instead.
- Never put verbs in the URL path — the HTTP method is the verb.

7. HTTP Methods (GET, POST, PUT, PATCH, DELETE)

Method	Purpose	Idempotent?	Has Request Body?
GET	Retrieve a resource or collection	Yes	No
POST	Create a new resource, or trigger a non-idempotent action	No	Yes
PUT	Fully replace an existing resource with the given representation	Yes	Yes
PATCH	Partially update specific fields of a resource	Not guaranteed (depends on implementation)	Yes
DELETE	Remove a resource	Yes	Usually no

7.1 Real Spring Boot Controller Using All Five

```
@RestController
@RequestMapping("/api/v1/accounts")
public class AccountsController {

    @GetMapping
    public List<AccountDto> getAllAccounts() { ... }

    @GetMapping("/{id}")
    public AccountDto getAccount(@PathVariable Long id) { ... }

    @PostMapping
    public ResponseEntity<AccountDto> createAccount(@Valid @RequestBody CreateAccountRequest request) {
        AccountDto created = accountsService.create(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }

    @PutMapping("/{id}")
    public AccountDto updateAccount(@PathVariable Long id, @Valid @RequestBody AccountDto fullUpdate) { ... }

    @PatchMapping("/{id}")
    public AccountDto patchAccount(@PathVariable Long id, @RequestBody Map<String, Object> updates) { ... }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> closeAccount(@PathVariable Long id) {
        accountsService.close(id);
        return ResponseEntity.noContent().build();
    }
}
```

Interview tip: PUT is idempotent because sending the same full replacement twice results in the same final state; POST is not idempotent because calling it twice typically creates two separate resources — this distinction is a very common interview question.

8. HTTP Status Codes — Full Real Reference

Code	Meaning	Real Use Case
200 OK	Success	Successful GET/PUT/PATCH
201 Created	Resource successfully created	Successful POST creating a new account
202 Accepted	Request accepted for async processing	Loan application submitted for background credit check
204 No Content	Success, no response body	Successful DELETE
400 Bad Request	Client sent invalid data	Validation failure — missing required field
401 Unauthorized	Missing or invalid authentication	No/expired JWT token provided
403 Forbidden	Authenticated but not authorized	A customer trying to access another customer's account
404 Not Found	Resource doesn't exist	GET /accounts/99999 (no such account)
409 Conflict	Request conflicts with current state	Trying to close an account that still has an active loan
422 Unprocessable Entity	Well-formed request but semantically invalid	Withdrawal amount exceeds available balance
429 Too Many Requests	Rate limit exceeded	Client exceeded allowed requests per minute
500 Internal Server Error	Unexpected server-side failure	Unhandled exception, bug in the code
502 Bad Gateway	Upstream service returned an invalid response	Gateway received garbage from a backend service
503 Service Unavailable	Server temporarily unable to handle the request	Service overloaded, or intentionally down for maintenance
504 Gateway Timeout	Upstream service didn't respond in time	Downstream call exceeded its timeout

Common mistake: Returning `200 OK` with an error message in the JSON body instead of an actual error status code — this breaks HTTP semantics and confuses clients, monitoring tools, and API Gateways that rely on status codes to detect failures.

9. Request & Response Design and the DTO Pattern

9.1 Why Not Expose Database Entities Directly?

Returning your JPA `@Entity` classes directly as API responses tightly couples your database schema to your public API contract — a database column rename breaks every client instantly, and you risk accidentally exposing sensitive fields (password hashes, internal flags) that were never meant to be public.

9.2 The DTO (Data Transfer Object) Pattern

Entity vs DTO — Separation of Concerns

Database Table (account)



`@Entity Account` (internal, JPA-managed)



mapped to →

`AccountDto` (public API contract)

```
@Entity
public class Account {
    @Id private Long id;
    private BigDecimal balance;
    private String internalRiskScore; // internal only – should NEVER be exposed via API
    private String passwordHash;    // definitely never exposed
}

// Public-facing DTO – only what the client actually needs
public record AccountDto(Long id, BigDecimal balance, String status) {}

@Service
public class AccountMapper {
    public AccountDto toDto(Account entity) {
        return new AccountDto(entity.getId(), entity.getBalance(), entity.getStatus());
    }
}
```

9.3 Request DTOs (Separate From Response DTOs)

```
// Separate request DTO – doesn't include server-generated fields like id/createdAt
public record CreateAccountRequest(
    @NotBlank String customerName,
    @NotNull @Positive BigDecimal initialDeposit
) {}
```

Using separate Request and Response DTOs (instead of one shared class) keeps validation rules clean and prevents clients from ever being able to set server-controlled fields like `id`, `createdAt`, or `status` directly in a create/update request.

10. API Versioning Strategies

10.1 Why Versioning Is Necessary

Once mobile apps, partner integrations, and other services depend on your API's exact shape, you can't just change it — versioning lets you evolve the API while existing consumers keep working on the old version until they can migrate.

10.2 Versioning Strategies Compared

Strategy	Example	Pros/Cons
URI Path Versioning	<code>/api/v1/accounts</code> , <code>/api/v2/accounts</code>	Most common, visible/discoverable, simple to test; "pollutes" the URL conceptually
Header Versioning	<code>Accept: application/vnd.bank.v2+json</code>	Clean URLs, but less discoverable, harder to test manually via browser
Query Parameter Versioning	<code>/accounts?version=2</code>	Simple but easy to misuse/forget, less strict

10.3 Real Practice — Running Two Versions in Parallel

```
@RestController
@RequestMapping("/api/v1/accounts")
public class AccountsControllerV1 { ... } // old shape, existing mobile app version still uses this

@RestController
@RequestMapping("/api/v2/accounts")
public class AccountsControllerV2 { ... } // new shape with additional fields, new mobile app version uses this
```

Real scenario: "We added a required `riskCategory` field to the account response. Instead of breaking the current mobile app release, we shipped it only in v2, migrated the mobile team over a few sprints, then deprecated v1 with a 6-month sunset notice via a `Deprecation` response header before finally removing it."

11. Pagination, Sorting & Filtering

11.1 Why Pagination Is Mandatory for Collections

Returning all 2 million transaction records in one response would crash the client, waste bandwidth, and slow the database to a crawl — any endpoint returning a list must support pagination from day one, not as an afterthought.

11.2 Real Spring Boot Pagination

```
@GetMapping("/accounts/{id}/transactions")
public Page<TransactionDto> getTransactions(
    @PathVariable Long id,
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "20") int size,
    @RequestParam(defaultValue = "transactionDate,desc") String sort) {

    Pageable pageable = PageRequest.of(page, size, Sort.by(parseSort(sort)));
    return transactionRepository.findById(id, pageable).map(mapper::toDto);
}

// Real request: GET /accounts/1001/transactions?page=0&size=20&sort=amount,desc
```

11.3 Real Paginated Response Shape

```
{
  "content": [ { "id": 1, "amount": 500.00 }, { "id": 2, "amount": 1200.00 } ],
  "pageNumber": 0,
  "pageSize": 20,
  "totalElements": 4820,
  "totalPages": 241,
  "last": false
}
```

11.4 Filtering — Real Query Parameter Design

```
# Filter transactions by date range and minimum amount
GET /accounts/1001/transactions?fromDate=2026-01-01&toDate=2026-01-31&minAmount=1000

@GetMapping("/accounts/{id}/transactions")
public Page<TransactionDto> getTransactions(
    @PathVariable Long id,
    @RequestParam(required = false) LocalDate fromDate,
    @RequestParam(required = false) LocalDate toDate,
    @RequestParam(required = false) BigDecimal minAmount,
    Pageable pageable) {
    return transactionService.search(id, fromDate, toDate, minAmount, pageable);
}
```

Use **Spring Data JPA Specifications** (dynamic **Criteria** queries) when the number of optional filter combinations grows large, instead of writing a separate repository method for every possible filter combination.

12. HATEOAS — Basic Understanding

12.1 What HATEOAS Means

HATEOAS (Hypermedia As The Engine Of Application State) is the idea that an API response should include links describing what actions/related resources are available next — so the client doesn't need to hardcode URL structures, it just follows links the server provides, similar to how a human browses a website by clicking links.

12.2 Real Example Response With HATEOAS Links

```
{
  "accountId": 1001,
  "balance": 4500.00,
  "status": "ACTIVE",
  "_links": {
    "self": { "href": "/api/v1/accounts/1001" },
    "transactions": { "href": "/api/v1/accounts/1001/transactions" },
    "close": { "href": "/api/v1/accounts/1001", "method": "DELETE" }
  }
}
```

12.3 Spring HATEOAS — Real Code

```
@GetMapping("/{id}")
public EntityModel<AccountDto> getAccount(@PathVariable Long id) {
    AccountDto account = accountsService.findById(id);
    return EntityModel.of(account,
        linkTo(methodOn(AccountsController.class).getAccount(id)).withSelfRel(),
        linkTo(methodOn(AccountsController.class).getTransactions(id)).withRel("transactions"));
}
```

12.4 Reality Check — How Often Is HATEOAS Actually Used?

In practice, most real-world REST APIs (including most internal microservices APIs) skip full HATEOAS — it adds complexity that many teams find isn't worth the benefit for APIs consumed by a frontend team that already knows the URL structure from documentation. It's more commonly seen in public APIs designed for long-term evolution by unknown third-party consumers, and is a good interview topic to understand conceptually even if you don't implement it everywhere.

13. Idempotency in REST APIs

13.1 Recap — What Idempotency Means

An operation is idempotent if calling it multiple times produces the same result/end-state as calling it once — critical because network retries and client double-clicks are a normal, expected occurrence, not an edge case.

13.2 Idempotency by HTTP Method (Built-In)

Method	Idempotent by Design?	Why
GET	Yes	Just reads data, no side effects
PUT	Yes	Replaces the full resource with the same given state every time
DELETE	Yes	Deleting an already-deleted resource still results in "it's gone"
POST	No (by default)	Typically creates a new resource each time — calling twice creates two resources

13.3 Making POST Idempotent — Idempotency Key Pattern

```
@PostMapping("/payments")
public ResponseEntity<PaymentResponse> makePayment(
    @RequestHeader("Idempotency-Key") String idempotencyKey,
    @RequestBody PaymentRequest request) {

    Optional<Payment> existing = paymentRepository.findByIdempotencyKey(idempotencyKey);
    if (existing.isPresent()) {
        return ResponseEntity.ok(toResponse(existing.get())); // return the SAME result, don't double-charge
    }
    Payment result = paymentService.process(request, idempotencyKey);
    return ResponseEntity.status(HttpStatus.CREATED).body(toResponse(result));
}
```

Real scenario: "A customer's mobile app timed out waiting for a payment response and auto-retried the exact same request. Because our payment API required an Idempotency-Key header (generated once client-side per logical payment attempt), the second request was recognized as a duplicate and returned the original result instead of charging the customer twice."

14. Error Handling & Global Exception Handling (Spring Boot)

14.1 Consistent Error Response Shape

Every error response across the entire API should follow the same JSON shape, so clients can write one generic error-handling routine instead of parsing different formats per endpoint.

```
{
  "timestamp": "2026-09-30T10:42:13Z",
  "status": 404,
  "error": "NOT_FOUND",
  "message": "Account with id 99999 not found",
  "path": "/api/v1/accounts/99999",
  "traceId": "a1b2c3d4"
}
```

14.2 Global Exception Handler — Real Spring Boot Code

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(AccountNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(AccountNotFoundException ex, HttpServletRequest req) {
        ErrorResponse error = ErrorResponse.of(HttpStatus.NOT_FOUND, ex.getMessage(), req.getRequestURI());
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(MethodArgumentNotValidException ex,
        HttpServletRequest req) {
        String message = ex.getBindingResult().getFieldErrors().stream()
            .map(f -> f.getField() + ": " + f.getDefaultMessage())
            .collect(Collectors.joining(", "));
        return ResponseEntity.badRequest().body(ErrorResponse.of(HttpStatus.BAD_REQUEST, message,
            req.getRequestURI()));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneric(Exception ex, HttpServletRequest req) {
        log.error("Unhandled exception", ex); // always log the full stack trace server-side
        ErrorResponse error = ErrorResponse.of(HttpStatus.INTERNAL_SERVER_ERROR,
            "An unexpected error occurred", req.getRequestURI()); // never leak internal details to the
        client
        return ResponseEntity.internalServerError().body(error);
    }
}
```

Security note: Never return raw exception messages or stack traces to the client for unexpected (500-level) errors — this can leak internal implementation details, file paths, or even SQL fragments useful to an attacker. Always log full details server-side and return a generic, safe message to the client.

15. Request Validation (Bean Validation)

15.1 Why Validate at the API Boundary

Validating input the moment it enters your system prevents bad data from ever reaching your business logic or database — fail fast, with a clear 400 Bad Request, rather than a confusing downstream error (or worse, silently corrupted data).

15.2 Real Bean Validation Annotations

```
public record CreateAccountRequest(
    @NotBlank(message = "Customer name is required")
    String customerName,

    @NotNull @Positive(message = "Initial deposit must be greater than zero")
    BigDecimal initialDeposit,

    @Email(message = "Invalid email format")
    String email,

    @Pattern(regexp = "^[0-9]{10}$", message = "Phone number must be 10 digits")
    String phoneNumber
) {}

@PostMapping
public ResponseEntity<AccountDto> createAccount(@Valid @RequestBody CreateAccountRequest request) {
    // @Valid triggers validation BEFORE this method body even runs – invalid requests never reach here
    return ResponseEntity.status(HttpStatus.CREATED).body(accountsService.create(request));
}
```

15.3 Custom Validators (Real Example — Business Rule Validation)

```
@Target(ElementType.FIELD)
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = ValidAccountTypeValidator.class)
public @interface ValidAccountType {
    String message() default "Invalid account type";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}

public class ValidAccountTypeValidator implements ConstraintValidator<ValidAccountType, String> {
    private static final Set<String> VALID_TYPES = Set.of("SAVINGS", "CURRENT", "SALARY");

    @Override
    public boolean isValid(String value, ConstraintValidatorContext context) {
        return VALID_TYPES.contains(value);
    }
}
```

16. OpenAPI / Swagger Deep Dive

16.1 What Is OpenAPI and What Is Swagger?

OpenAPI is the formal **specification** (a YAML/JSON document describing every endpoint, request/response shape, and auth requirement of an API). Swagger is the toolset built around that specification — Swagger UI (interactive documentation browser), Swagger Codegen (generates client SDKs), and springdoc-openapi (auto-generates the spec from your Spring Boot annotations).

16.2 Why Swagger/OpenAPI Matters So Much in Real Organizations

- **Living, always-accurate documentation** — generated directly from the actual code/annotations, so it can't drift out of sync like a manually maintained Word doc.
- **Interactive testing** — Swagger UI lets any developer (or QA, or a partner team) try real API calls directly from the browser, no Postman collection needed.
- **Client SDK generation** — automatically generate typed client code for React/TypeScript, other Java services, or mobile apps directly from the spec.
- **Contract-first development** — teams can agree on the OpenAPI spec before either side writes implementation code (Section 38).

16.3 Real Spring Boot Setup — springdoc-openapi

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.5.0</version>
</dependency>

# Once added, Swagger UI is automatically available at:
# http://localhost:8081/swagger-ui.html
# Raw OpenAPI JSON spec available at:
# http://localhost:8081/v3/api-docs
```

16.4 Documenting Endpoints With Real Annotations

```
@RestController
@RequestMapping("/api/v1/accounts")
@Tag(name = "Accounts", description = "Operations for managing customer bank accounts")
public class AccountsController {

    @Operation(summary = "Get account by ID", description = "Returns full account details for a given account ID")
    @ApiResponse({
        @ApiResponse(responseCode = "200", description = "Account found",
            content = @Content(schema = @Schema(implementation = AccountDto.class))),
        @ApiResponse(responseCode = "404", description = "Account not found",
            content = @Content(schema = @Schema(implementation = ErrorResponse.class)))
    })
    @GetMapping("/{id}")
    public AccountDto getAccount(
        @Parameter(description = "The unique account identifier", example = "1001")
        @PathVariable Long id) {
        return accountsService.findById(id);
    }
}
```

16.5 Global OpenAPI Metadata & Security Scheme Configuration

```
@Configuration
public class OpenApiConfig {
    @Bean
    public OpenAPI bankApiOpenAPI() {
        return new OpenAPI()
            .info(new Info().title("Accounts Service API").version("v1")
                .description("Real-time banking Accounts microservice API"))
            .addSecurityItem(new SecurityRequirement().addList("bearerAuth"))
            .components(new Components().addSecuritySchemes("bearerAuth",
                new SecurityScheme().type(SecurityScheme.Type.HTTP).scheme("bearer").bearerFormat("JWT")));
    }
}
```

16.6 Example OpenAPI YAML Snippet (Generated Spec)

```
openapi: 3.0.1
info:
  title: Accounts Service API
  version: v1
paths:
  /api/v1/accounts/{id}:
    get:
      summary: Get account by ID
      parameters:
        - name: id
          in: path
          required: true
          schema: { type: integer, format: int64 }
      responses:
        '200':
          description: Account found
          content:
            application/json:
              schema: { $ref: '#/components/schemas/AccountDto' }
        '404':
          description: Account not found
```

Real scenario: "Our organization treats the auto-generated Swagger UI as the single source of truth for API documentation — new team members onboard by exploring Swagger UI directly, QA writes test cases straight from it, and our frontend team generates a fully typed TypeScript API client from the OpenAPI spec automatically as part of the CI pipeline, so the frontend never manually writes fetch calls with hand-typed response shapes."

17. Rate Limiting for APIs

17.1 Why Rate Limit an API

Protects backend services from being overwhelmed — whether from a buggy client stuck in a retry loop, a genuine traffic spike, or a malicious/abusive client — and ensures fair usage across all consumers of a shared API.

17.2 Real Bucket4j-Based Rate Limiting in Spring Boot

```
@Component
public class RateLimitFilter extends OncePerRequestFilter {
    private final Map<String, Bucket> buckets = new ConcurrentHashMap<>();

    private Bucket newBucket() {
        Bandwidth limit = Bandwidth.classic(100, Refill.greedy(100, Duration.ofMinutes(1)));
        return Bucket.builder().addLimit(limit).build();
    }

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
        throws ServletException, IOException {
        String apiKey = req.getHeader("X-API-Key");
        Bucket bucket = buckets.computeIfAbsent(apiKey, k -> newBucket());
        if (bucket.tryConsume(1)) {
            chain.doFilter(req, res);
        } else {
            res.setStatus(429);
            res.getWriter().write("{\"error\":\"Rate limit exceeded, try again later\"}");
        }
    }
}
```

17.3 Rate Limiting at the API Gateway Level (Preferred in Real Orgs)

Most organizations enforce rate limiting centrally at the API Gateway (Spring Cloud Gateway, Kong, Azure API Management) rather than in every individual service — one consistent policy, applied once, protecting every backend service uniformly.

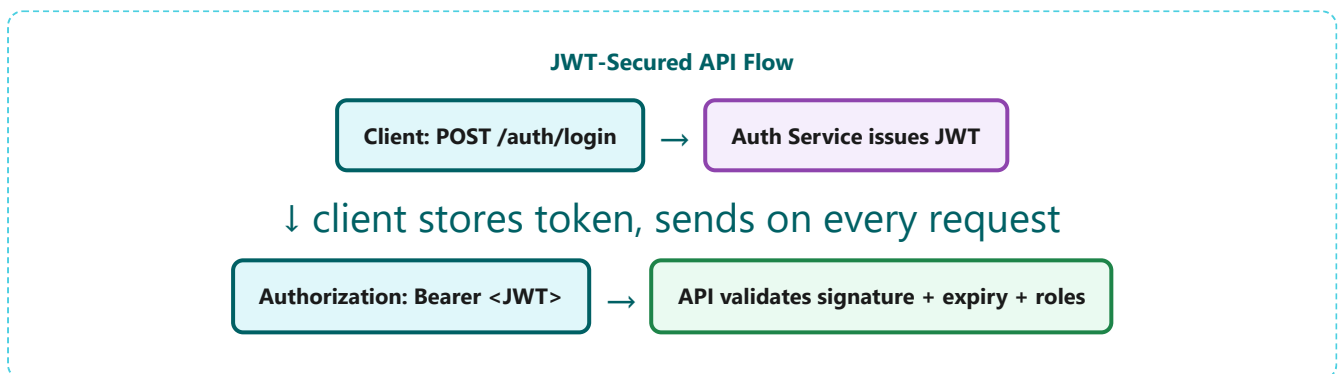
```
spring:
  cloud:
    gateway:
      routes:
        - id: accounts-route
          uri: lb://ACCOUNTS-SERVICE
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 50
                redis-rate-limiter.burstCapacity: 100
```

18. API Security — JWT Authentication

18.1 JWT Recap for APIs

A JWT (JSON Web Token) is a signed, self-contained token carrying claims (user ID, roles, expiry) — the API validates its signature and expiry on every request without needing a server-side session store, making it ideal for stateless REST APIs.

18.2 Real JWT Flow for a REST API



18.3 Real Spring Security Resource Server Config

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/v1/auth/**", "/swagger-ui/**", "/v3/api-docs/**").permitAll()
                .requestMatchers(HttpMethod.DELETE, "/api/v1/accounts/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .oauth2ResourceServer(oauth2 -> oauth2.jwt(Customizer.withDefaults()));
        return http.build();
    }
}
```

18.4 Extracting the Current User From the JWT

```
@GetMapping("/me/accounts")
public List<AccountDto> getMyAccounts(@AuthenticationPrincipal Jwt jwt) {
    String customerId = jwt.getClaimAsString("customerId");
    return accountsService.findByCustomerId(customerId);
}
```

19. API Security — OAuth2

19.1 OAuth2 Roles Recap

Role	Meaning
Resource Owner	The end user who owns the protected data
Client	The application requesting access on the user's behalf (React app, mobile app)
Authorization Server	Issues tokens after authenticating the user (Keycloak, Azure AD, Okta)
Resource Server	The API that hosts protected data and validates tokens (our Spring Boot services)

19.2 OAuth2 Grant Types — When to Use Which

Grant Type	Used For
Authorization Code (+ PKCE)	Standard web/mobile app login flow — most common and most secure for user-facing apps
Client Credentials	Service-to-service (machine-to-machine) authentication — no user involved
Refresh Token	Obtaining a new access token without forcing the user to log in again
Implicit (deprecated)	Legacy, no longer recommended — replaced by Authorization Code + PKCE even for SPAs

19.3 Client Credentials Flow — Real Service-to-Service Example

```
# Loans Service calling an external Credit Bureau API using Client Credentials grant
POST https://auth.creditbureau.com/oauth/token
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials&client_id=bank-loans-service&client_secret=***&scope=credit.read

# Response
{ "access_token": "eyJhbGciOiAi... ", "token_type": "Bearer", "expires_in": 3600 }
```

Interview tip: JWT is a **token format**; OAuth2 is an **authorization framework** that defines how tokens (often JWTs) are issued and used. You can have OAuth2 without JWT (opaque tokens validated via introspection) and JWTs without OAuth2 (a custom auth system) — but in practice, most modern systems combine both.

20. CORS Explained

20.1 What CORS Is and Why It Exists

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that blocks a web page from calling an API hosted on a different origin (domain/port/protocol) unless that API explicitly allows it — this is a **browser-enforced** rule, not a server-side security feature (Postman/curl/mobile apps are never blocked by CORS).

20.2 Real CORS Error (What Developers See)

```
// Browser console error when CORS isn't configured
Access to fetch at 'https://api.mybank.com/accounts' from origin 'https://app.mybank.com'
has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present
```

20.3 Real Spring Boot CORS Configuration

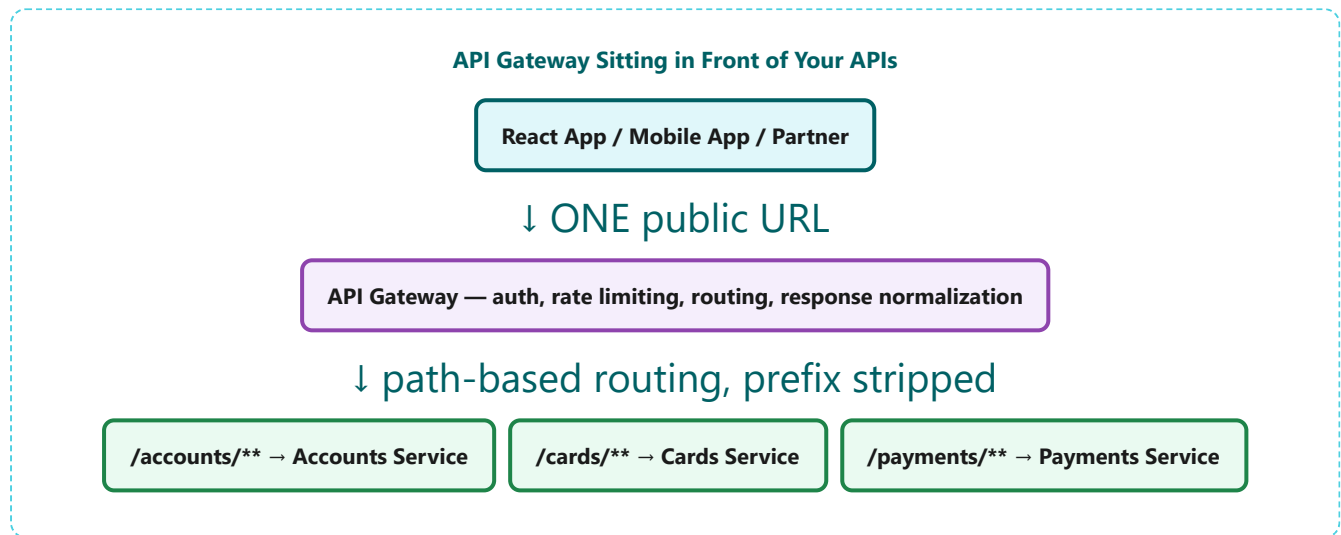
```
@Configuration
public class CorsConfig {
    @Bean
    public WebMvcConfigurer corsConfigurer() {
        return new WebMvcConfigurer() {
            @Override
            public void addCorsMappings(CorsRegistry registry) {
                registry.addMapping("/api/**")
                    .allowedOrigins("https://app.mybank.com")    // never use "*" with credentials in
production
                    .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE")
                    .allowedHeaders("Authorization", "Content-Type")
                    .allowCredentials(true)
                    .maxAge(3600);
            }
        };
    }
}
```

Common mistake: Setting `allowedOrigins("*")` alongside `allowCredentials(true)` — browsers explicitly reject this combination for security reasons; you must specify exact allowed origins when credentials (cookies/auth headers) are involved.

21. API Gateway Integration

21.1 Why APIs Sit Behind a Gateway

Rather than exposing every microservice's API directly to the internet, a single API Gateway becomes the one entry point handling routing, authentication, rate limiting, and cross-cutting concerns centrally — full details already covered in the Microservices guide (Section 12); this section focuses specifically on the API-design implications.



21.2 What Changes in API Design When a Gateway Is in Front

- Individual services can trust that authentication already happened at the Gateway (though defense-in-depth means re-validating the token is still good practice).
- Path-based routing means services should design URLs assuming a prefix will be stripped/rewritten (e.g., Gateway strips `/accounts` before forwarding to the actual Accounts Service).
- Response aggregation — a Gateway (or a Backend-for-Frontend layer) can combine multiple service calls into one response for the frontend, reducing round trips.
- Consistent error format enforcement — the Gateway can normalize error responses even if individual services aren't perfectly consistent.

22. Logging & Auditing for APIs

22.1 Access Logging — Every Request

```
@Component
public class ApiLoggingFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
        throws ServletException, IOException {
        long start = System.currentTimeMillis();
        chain.doFilter(req, res);
        long duration = System.currentTimeMillis() - start;
        log.info("method={ } path={ } status={ } durationMs={ } traceId={ }",
            req.getMethod(), req.getRequestURI(), res.getStatus(), duration, MDC.get("traceId"));
    }
}
```

22.2 Audit Logging — Who Did What (Compliance-Critical)

```
@Around("@annotation(Auditable)")
public Object auditAction(ProceedingJoinPoint joinPoint) throws Throwable {
    String user = SecurityContextHolder.getContext().getAuthentication().getName();
    Object result = joinPoint.proceed();
    auditLogRepository.save(new AuditLog(user, joinPoint.getSignature().getName(), Instant.now()));
    return result;
}

@Auditable
@PatchMapping("/{cardId}/limit")
public CardDto updateCardLimit(@PathVariable Long cardId, @RequestBody UpdateLimitRequest request) { ... }
```

Access logs answer "what happened technically" (for debugging/monitoring); Audit logs answer "who changed what business data, when" (for compliance/security reviews) — banking/financial APIs typically require both, stored separately with different retention policies.

23. API Documentation Best Practices

- **Generate docs from code (Swagger/OpenAPI)**, never maintain a separate hand-written Word/Confluence doc that will inevitably drift out of sync.
- **Document every possible response**, including error responses (400/404/409), not just the happy path 200.
- **Provide real, working examples** for both requests and responses — abstract schemas alone are hard to use quickly.
- **Document authentication requirements clearly** per endpoint (which roles/scopes are needed).
- **Version the documentation alongside the API** — old API versions should still have their docs available until fully retired.
- **Include rate limits and pagination conventions** in a general "Getting Started" section, not repeated per endpoint.
- **Keep a changelog** of breaking vs non-breaking changes per release, so consumers know what to check before upgrading.

24. REST API Standards — Naming Conventions & Consistent Response Structure

24.1 Consistent Naming Conventions

Rule	Example
Plural nouns for collections	<code>/accounts</code> , not <code>/account</code>
Lowercase, hyphenated paths	<code>/loan-applications</code> , not <code>/loanApplications</code>
camelCase for JSON field names	<code>customerName</code> , not <code>customer_name</code> (pick one convention org-wide and stick to it)
Consistent date/time format	ISO-8601 everywhere: <code>2026-09-30T10:42:13Z</code>
Consistent ID types	Always numeric IDs, or always UUIDs — never mixed across services

24.2 Standard Response Envelope (Real Organization Pattern)

```
// Successful response
{
  "success": true,
  "data": { "accountId": 1001, "balance": 4500.00 },
  "traceId": "a1b2c3d4"
}

// Error response – same envelope shape, different content
{
  "success": false,
  "error": { "code": "ACCOUNT_NOT_FOUND", "message": "Account 99999 not found" },
  "traceId": "a1b2c3d4"
}
```

A consistent top-level envelope (even if simple) means every frontend/consumer can write one generic response-handling function instead of custom parsing per endpoint — small consistency decisions like this save enormous integration pain across dozens of consuming teams.

25. Correlation ID & Trace ID Propagation

Full conceptual detail is covered in the Observability & Monitoring guide (Section 9) — this section focuses on the API-specific implementation: every API must accept an incoming **X-Correlation-Id** / **traceparent** header if present, generate one if absent, include it in every log line, and propagate it to every downstream call it makes.

```
@Component
public class CorrelationIdFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
        throws ServletException, IOException {
        String correlationId = Optional.ofNullable(req.getHeader("X-Correlation-Id"))
            .orElse(UUID.randomUUID().toString());
        MDC.put("correlationId", correlationId);
        res.setHeader("X-Correlation-Id", correlationId); // echo back so the client can also log/report it
        try { chain.doFilter(req, res); } finally { MDC.clear(); }
    }
}

// Feign automatically propagates it to downstream calls via a RequestInterceptor
@Bean
public RequestInterceptor correlationIdInterceptor() {
    return template -> template.header("X-Correlation-Id", MDC.get("correlationId"));
}
```

26. Response Compression

Compressing API responses (typically with gzip) reduces bandwidth usage and improves perceived performance for clients on slower networks (especially mobile), at a small CPU cost on the server to compress each response.

```
server:
  compression:
    enabled: true
    mime-types: application/json, application/xml
    min-response-size: 1024  # only compress responses larger than 1KB - not worth it for tiny payloads
```

Consideration	Note
Client support	Client must send <code>Accept-Encoding: gzip</code> — virtually all modern browsers/HTTP clients do this automatically
CPU tradeoff	Compression uses CPU on the server; usually a good tradeoff for JSON-heavy APIs at scale
Already-compressed data	Don't bother compressing already-compressed payloads (images, zipped files)

27. API Backward Compatibility

27.1 What Counts as a Breaking Change

Change Type	Breaking?	Why
Adding a new optional field to a response	No	Existing clients simply ignore fields they don't recognize
Removing a field from a response	Yes	Clients relying on that field will break/error
Renaming a field	Yes	Functionally the same as removing one field and adding a new one
Adding a new optional request parameter	No	Existing clients simply don't send it, default behavior applies
Making a previously optional request field required	Yes	Existing clients not sending it will now fail validation
Changing a field's data type	Yes	Client-side parsing/deserialization will likely fail or misbehave
Changing an error code/status for the same scenario	Yes	Clients with specific error-handling logic per status code will behave incorrectly

27.2 Practices for Maintaining Backward Compatibility

- Always add new fields as optional, never remove or repurpose existing ones without a version bump.
- Use API versioning (Section 10) for any genuinely breaking change.
- Use Consumer-Driven Contract testing (Pact — Section 38) in CI to automatically catch accidental breaking changes before release.
- Communicate deprecations clearly via response headers (`Deprecation` , `Sunset`) and documentation well ahead of actually removing anything.

28. Retry Strategy, Timeout Handling & Circuit Breaker for APIs

28.1 Timeout — The Baseline Every API Call Needs

```
@Bean
public WebClient webClient(WebClient.Builder builder) {
    HttpClient httpClient = HttpClient.create()
        .responseTimeout(Duration.ofSeconds(3))           // never wait indefinitely on a downstream API
        .option(ChannelOption.CONNECT_TIMEOUT_MILLIS, 2000);
    return builder.clientConnector(new ReactorClientHttpConnector(httpClient)).build();
}
```

28.2 Retry With Backoff (When Calling Another API)

```
@Retry(name = "creditBureauApi", fallbackMethod = "fallbackCreditScore")
public CreditScoreResponse getCreditScore(String customerId) {
    return creditBureauClient.getScore(customerId);
}

resilience4j:
  retry:
    instances:
      creditBureauApi:
        max-attempts: 3
        wait-duration: 500ms
        exponential-backoff-multiplier: 2
        retry-exceptions: [ java.io.IOException, java.util.concurrent.TimeoutException ]
```

28.3 Circuit Breaker for Downstream API Calls

Full pattern detail is covered in the Microservices guide (Section 18) — for APIs specifically, wrap every outbound call to another internal service or third-party API in a circuit breaker with a sensible fallback, so one struggling dependency never cascades into your own API becoming unresponsive.

29. Caching Strategy for APIs

29.1 HTTP-Level Caching Headers

```
@GetMapping("/products/interest-rates")
public ResponseEntity<List<InterestRateDto>> getInterestRates() {
    List<InterestRateDto> rates = productService.getCurrentRates();
    return ResponseEntity.ok()
        .cacheControl(CacheControl.maxAge(1, TimeUnit.HOURS).cachePublic())    # tells browsers/CDNs to cache
        for 1 hour
        .eTag(Integer.toString(rates.hashCode()))    # lets clients do conditional GETs (304 Not Modified)
        .body(rates);
}
```

Header	Purpose
Cache-Control: max-age=3600	Tells clients/CDNs how long the response is considered fresh
ETag	A version fingerprint — client sends it back via If-None-Match ; server returns 304 if unchanged, saving bandwidth
Cache-Control: no-store	Explicitly forbids caching — for sensitive data like account balances

29.2 Server-Side Caching (Redis) Recap

Full detail in the Microservices guide (Section 35) — for read-heavy, rarely-changing API responses (product catalogs, interest rate tables), cache the actual data server-side in Redis so repeated API calls don't hit the database every time, while explicitly never caching real-time sensitive data like current balances.

30. Monitoring & Observability for APIs

Full detail is covered in the dedicated Observability & Monitoring guide — for APIs specifically, the essentials are: per-endpoint request rate/error rate/latency percentiles (not just service-wide averages), status code breakdown (4xx vs 5xx trends), and distributed tracing across every downstream call an API makes.

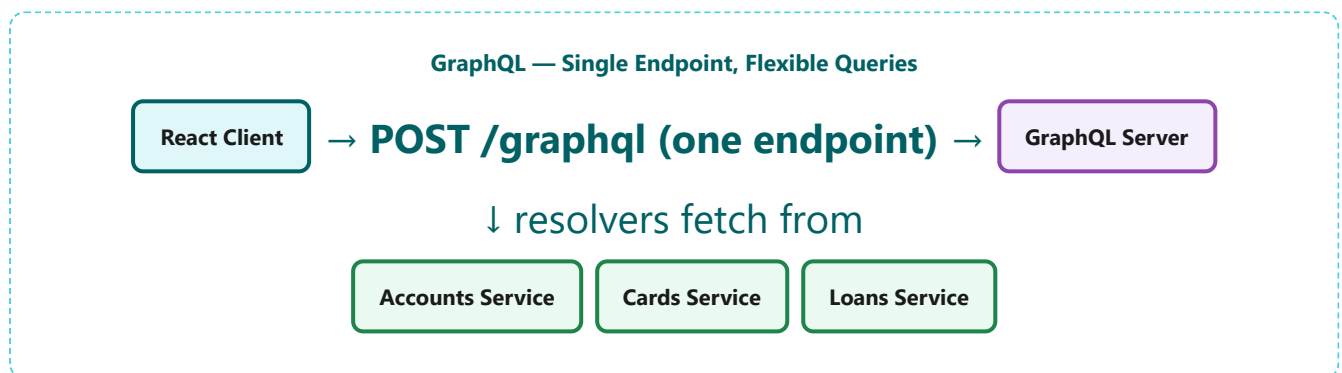
What to Monitor Per API Endpoint	Why
Request count & rate	Understand real usage patterns, capacity planning
Error rate by status code	4xx spikes often mean a client bug/contract issue; 5xx spikes mean a real backend problem
Latency percentiles (p50/p95/p99)	Averages hide the worst-case experience — always look at p95/p99
Payload size	Unexpectedly large responses can indicate a missing pagination bug

31. GraphQL — Architecture, Schema, Query, Mutation, Subscription

31.1 What Problem GraphQL Solves

Over-fetching: a REST `/accounts/1001` call might return 20 fields when the UI only needs 3. **Under-fetching:** the UI needs data from 3 different REST endpoints (account, cards, loans) requiring 3 separate round trips. GraphQL solves both — the client specifies exactly which fields it needs, from potentially multiple underlying sources, in one single request.

31.2 GraphQL Architecture



31.3 Schema Design — The Contract

```
type Account {
  id: ID!
  balance: Float!
  status: String!
  cards: [Card!]!      # resolved by calling Cards Service under the hood
  loans: [Loan!]!     # resolved by calling Loans Service under the hood
}

type Card {
  id: ID!
  last4Digits: String!
  limit: Float!
}

type Query {
  account(id: ID!): Account
}

type Mutation {
  closeAccount(id: ID!): Account
}

type Subscription {
  accountBalanceChanged(id: ID!): Account
}
```

31.4 Query — Real Example

```
# Client asks for EXACTLY the fields it needs, nothing more, in ONE request
query {
  account(id: "1001") {
    balance
    status
    cards {
      last4Digits
    }
  }
}

# Response mirrors the exact shape requested
{ "data": { "account": { "balance": 4500.00, "status": "ACTIVE", "cards": [ { "last4Digits": "4321" } ] } } }
```

31.5 Mutation — Real Example (Writes)

```
mutation {
  closeAccount(id: "1001") {
    id
    status
  }
}
```

31.6 Subscription — Real-Time Updates

Subscriptions use a persistent connection (typically WebSocket) to push updates to the client whenever a specific event occurs server-side — e.g., notifying a dashboard the instant an account's balance changes, without polling.

31.7 Real Spring Boot GraphQL Setup

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-graphql</artifactId>
</dependency>

@Controller
public class AccountGraphQLController {

    @QueryMapping
    public Account account(@Argument String id) {
        return accountsService.findById(Long.parseLong(id));
    }

    @MutationMapping
    public Account closeAccount(@Argument String id) {
        return accountsService.close(Long.parseLong(id));
    }
}
```

32. GraphQL Resolvers, Data Fetching, the N+1 Problem & DataLoader

32.1 What Is a Resolver?

A resolver is a function that knows how to fetch the data for one specific field in the schema — when a client requests `account { cards { last4Digits } }`, one resolver fetches the account, and a separate resolver fetches the cards for that account.

32.2 The N+1 Problem — A Real Trap

If a client queries a list of 50 accounts, each including their cards, a naive implementation runs 1 query for the accounts, then **50 separate queries** (one per account) to fetch each account's cards — 51 total queries instead of 2, an easy-to-miss but serious performance trap unique to GraphQL's field-by-field resolution model.

N+1 Problem Visualized

1 query: fetch 50 accounts

↓ then, naively, PER account:

50 separate queries: fetch cards for account #1, #2, #3 ... #50

32.3 DataLoader — The Solution

DataLoader batches and deduplicates data-fetching calls within a single request tick — instead of firing 50 individual "fetch cards for account X" calls, it collects all 50 account IDs and fires **one** batched "fetch cards for these 50 account IDs" call.

```
@Bean
public DataLoader<Long, List<Card>> cardsDataLoader(CardsServiceClient cardsClient) {
    return DataLoader.newMappedDataLoader(accountIds ->
        CompletableFuture.supplyAsync(() -> cardsClient.getCardsForAccounts(accountIds)) // ONE batched call
    );
}

@SchemaMapping(typeName = "Account", field = "cards")
public CompletableFuture<List<Card>> cards(Account account, DataLoader<Long, List<Card>> cardsLoader) {
    return cardsLoader.load(account.getId()); // batched automatically behind the scenes
}
```

Interview tip: The N+1 problem isn't unique to GraphQL (it also happens with JPA lazy loading in REST APIs — Section 43), but GraphQL's flexible, nested query model makes it much easier to accidentally trigger at scale, which is exactly why DataLoader is considered a near-mandatory pattern in any real GraphQL implementation.

33. GraphQL Security, Query Complexity, Pagination & Federation

33.1 GraphQL Security Concerns

Because GraphQL exposes one flexible endpoint, a malicious or careless client can construct extremely deep, nested, or expensive queries (e.g., account → cards → transactions → account → cards → ... nested many levels) that overload the server — a risk REST's fixed-endpoint model doesn't share in the same way.

33.2 Query Complexity / Depth Limiting

```
# Real protections applied at the GraphQL server level:  
# - Max query depth limit (e.g., reject queries nested more than 6 levels deep)  
# - Query cost analysis (assign a "cost" per field, reject queries exceeding a total budget)  
# - Timeouts per query execution  
# - Persisted queries (only pre-approved, whitelisted queries allowed in production)
```

33.3 Pagination in GraphQL — Cursor-Based (Relay-Style)

```
type Query {  
  transactions(first: Int, after: String): TransactionConnection!  
}  
  
type TransactionConnection {  
  edges: [TransactionEdge!]!  
  pageInfo: PageInfo!  
}  
  
type PageInfo {  
  hasNextPage: Boolean!  
  endCursor: String  
}
```

Cursor-based pagination (rather than simple page numbers) is preferred in GraphQL because it handles data that changes between page requests more gracefully (no skipped/duplicated items when new records are inserted mid-pagination).

33.4 Error Handling in GraphQL

Unlike REST, GraphQL almost always returns HTTP 200 even for application-level errors — errors are reported inside a dedicated `errors` array in the response body alongside any partially successful `data`, which is a notable and frequently-asked interview difference from REST.

```
{
  "data": { "account": null },
  "errors": [ { "message": "Account not found", "path": ["account"], "extensions": { "code": "NOT_FOUND" } } ]
}
```

33.5 GraphQL Federation

Federation lets multiple teams each own a piece of one unified GraphQL schema — Accounts team owns the **Account** type, Cards team owns the **Card** type (with a reference back to Account) — and a Gateway composes them into a single graph, mirroring microservice ownership boundaries without forcing one team to own the entire schema.

34. GraphQL vs REST — Full Comparison

Aspect	REST	GraphQL
Endpoints	Multiple, one per resource	Single endpoint (<code>/graphql</code>)
Data fetching	Fixed response shape per endpoint — risk of over/under-fetching	Client specifies exactly the fields needed — no over/under-fetching
Versioning	Explicit versions (v1, v2)	Typically evolves the schema additively, avoiding hard version bumps
Caching	Simple, leverages standard HTTP caching (Cache-Control, ETag, CDNs)	Harder — single endpoint, POST-based, requires custom caching solutions
Error handling	Standard HTTP status codes	Almost always 200 OK, errors reported in the response body
File uploads	Native, simple (multipart/form-data)	Requires extensions/workarounds, not natively simple
Learning curve	Lower, widely understood	Higher — schema design, resolvers, N+1 awareness required
Best fit	Most public/internal APIs, simple CRUD services, when HTTP caching matters	Complex UIs (dashboards) aggregating many data sources, mobile apps minimizing over-the-wire data

Real scenario: "Our core banking microservices all expose REST APIs (simple, cacheable, easy to secure per-service). Our single customer-facing 'Dashboard' team built one GraphQL layer on top of them specifically to let the frontend fetch a customer's accounts + cards + loans summary in one request, instead of the frontend making three separate REST calls and manually merging the results."

35. gRPC — Protocol Buffers & Service Definitions

35.1 What Is gRPC?

gRPC is a high-performance RPC (Remote Procedure Call) framework from Google using **Protocol Buffers** (a compact binary serialization format) over HTTP/2 — significantly faster and smaller over the wire than JSON/REST, with a strongly-typed contract shared between client and server.

35.2 Protocol Buffers (.proto) — Real Definition

```
syntax = "proto3";

package bank.loans;

service LoansService {
  rpc GetLoanDetails (LoanRequest) returns (LoanResponse);
  rpc StreamRepaymentSchedule (LoanRequest) returns (stream RepaymentEntry);
}

message LoanRequest {
  int64 loanId = 1;
}

message LoanResponse {
  int64 loanId = 1;
  double outstandingAmount = 2;
  string status = 3;
}

message RepaymentEntry {
  int32 installmentNumber = 1;
  double amount = 2;
  string dueDate = 3;
}
```

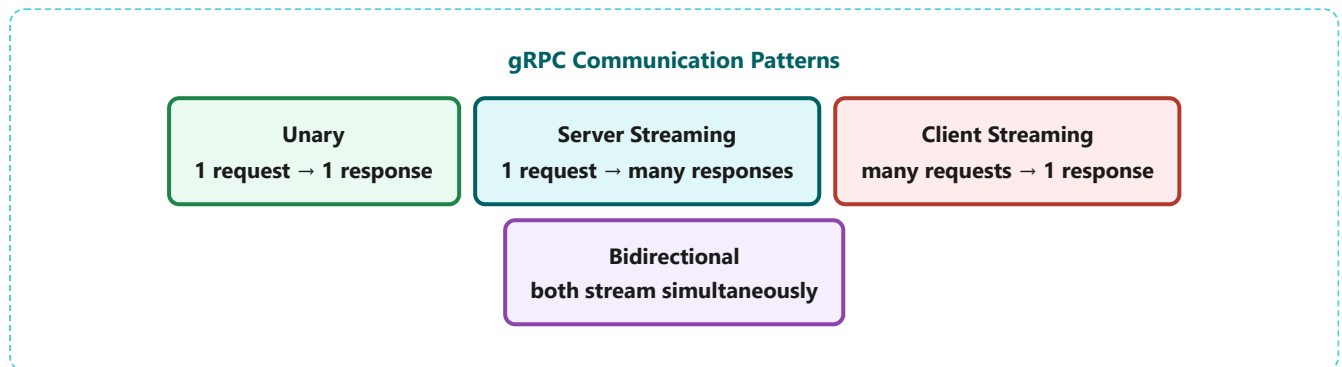
Every field has an explicit number (e.g., `= 1`, `= 2`) — these numbers, not the field names, are what's actually encoded on the wire, making Protobuf messages extremely compact compared to JSON's repeated text field names.

35.3 Stub Generation

The `.proto` file is compiled (via `protoc`) into strongly-typed client "stub" and server "skeleton" code in your target language (Java, Python, Go, etc.) — both sides share the exact same generated contract, eliminating manual JSON parsing/serialization entirely and catching contract mismatches at compile time, not runtime.

36. gRPC Call Types — Unary, Server Streaming, Client Streaming, Bidirectional

36.1 The Four Call Types



36.2 Real Examples for Each

Type	Real Banking Use Case
Unary	"Get loan details for loan ID 4521" — a simple single request/response, most common type
Server Streaming	"Stream all 60 repayment installments for this loan" — server sends a continuous stream of entries
Client Streaming	"Upload a batch of transaction records for reconciliation" — client streams many records, server responds once with a summary
Bidirectional Streaming	A live fraud-detection feed — client streams transactions as they happen, server streams risk scores back in real time

36.3 Real Java Unary Call Code

```
LoansServiceGrpc.LoansServiceBlockingStub stub = LoansServiceGrpc.newBlockingStub(channel);
LoanRequest request = LoanRequest.newBuilder().setLoanId(4521).build();
LoanResponse response = stub.getLoanDetails(request);
System.out.println("Outstanding: " + response.getOutstandingAmount());
```

37. gRPC Deep Dive — Serialization, Stubs, Deadlines, Auth, TLS, Interceptors

37.1 Deadlines & Timeouts (gRPC's Built-In Timeout Concept)

```
LoanResponse response = stub
    .withDeadlineAfter(2, TimeUnit.SECONDS) // gRPC's native deadline propagation
    .getLoanDetails(request);
```

gRPC deadlines propagate automatically across chained calls — if Service A calls Service B with a 2-second deadline, and B calls C, the remaining time budget is passed along, so C doesn't keep working after A has already given up waiting — a more elegant built-in mechanism than manually re-implementing timeouts at every hop like REST typically requires.

37.2 Authentication & TLS

```
// TLS-secured channel with token-based auth metadata attached to every call
ManagedChannel channel = ManagedChannelBuilder.forAddress("loans-service", 443)
    .useTransportSecurity() // TLS enabled – encrypts traffic in transit
    .build();

Metadata metadata = new Metadata();
metadata.put(Metadata.Key.of("authorization", Metadata.ASCII_STRING_MARSHALLER), "Bearer " + jwtToken);
```

37.3 Retry Policies (Built Into the Channel Config)

```
{
  "methodConfig": [{
    "name": [{ "service": "bank.loans.LoansService" }],
    "retryPolicy": {
      "maxAttempts": 3,
      "initialBackoff": "0.5s",
      "maxBackoff": "5s",
      "backoffMultiplier": 2,
      "retryableStatusCodes": ["UNAVAILABLE", "DEADLINE_EXCEEDED"]
    }
  }]
}
```

37.4 Interceptors (gRPC's Equivalent of Servlet Filters)

```
public class LoggingInterceptor implements ServerInterceptor {  
    @Override  
    public <ReqT, RespT> ServerCall.Listener<ReqT> interceptCall(  
        ServerCall<ReqT, RespT> call, Metadata headers, ServerCallHandler<ReqT, RespT> next) {  
        log.info("gRPC call: {}", call.getMethodDescriptor().getFullName());  
        return next.startCall(call, headers);  
    }  
}
```

37.5 Error Handling in gRPC

gRPC uses standardized status codes (distinct from HTTP status codes) — e.g., `NOT_FOUND`, `UNAUTHENTICATED`, `DEADLINE_EXCEEDED`, `UNAVAILABLE` — returned alongside an optional descriptive message, giving clients a consistent, language-agnostic way to handle failures programmatically.

38. gRPC vs REST — When to Use Which

Aspect	REST (JSON/HTTP)	gRPC (Protobuf/HTTP2)
Performance	Slower — text-based, typically HTTP/1.1	Much faster — binary, HTTP/2 multiplexing
Payload size	Larger (verbose JSON field names repeated)	Much smaller (binary, field numbers not names)
Human readability	High — easy to inspect with curl/Postman/browser	Low — binary, needs special tooling (grpcurl, BloomRPC)
Browser support	Native	Needs a grpc-web proxy layer for browser clients
Streaming support	Limited (requires workarounds like SSE/WebSocket)	Native, first-class streaming in all 4 directions
Contract strictness	Flexible (OpenAPI optional)	Strict, strongly-typed .proto contract required
Best fit	Public APIs, browser-facing endpoints, anything needing HTTP caching	High-throughput internal service-to-service calls, streaming use cases

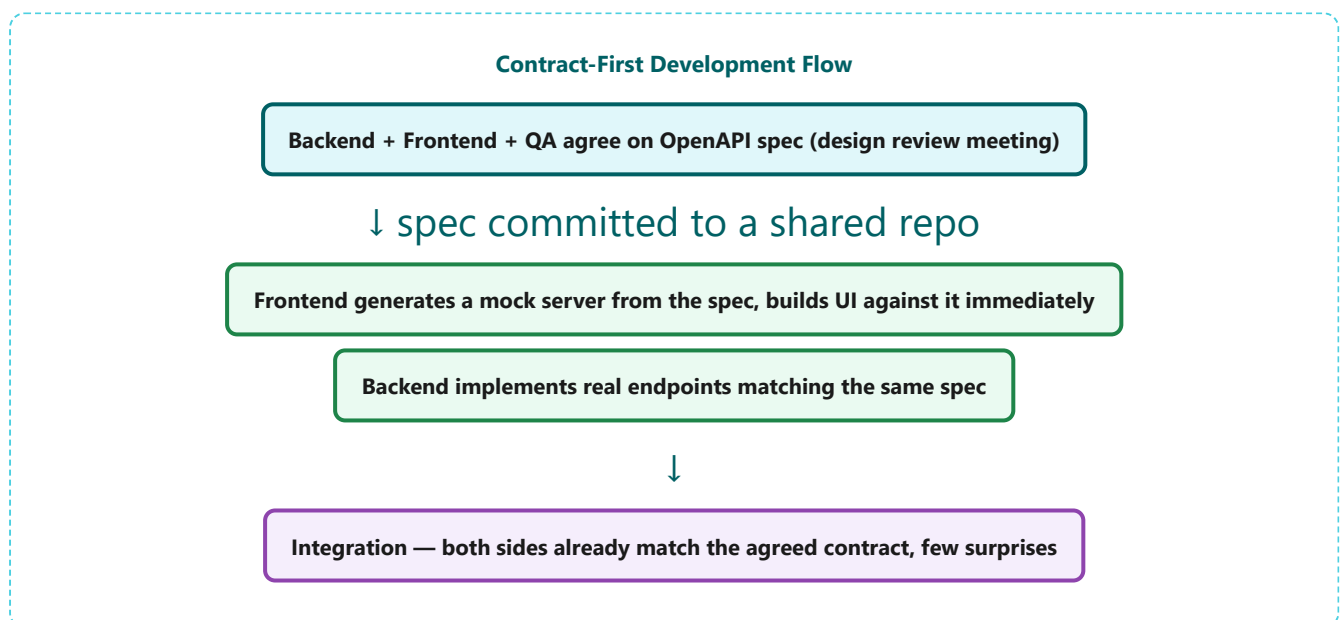
Real-world note (recap from the Microservices guide): Most organizations default to REST for anything customer/browser-facing (caching, tooling, human-readability all matter there), and reserve gRPC for specific high-volume internal paths where the performance and native streaming benefits clearly justify the added tooling complexity.

39. Real-Time API Design — Contract-First & Consumer-Driven Contracts

39.1 API-First / Contract-First Development

Instead of writing backend code first and letting documentation follow, contract-first development means writing the OpenAPI/Protobuf/GraphQL schema **first**, agreeing on it across teams (frontend, backend, QA, partner teams), and only then implementing against that agreed contract — enabling parallel work since frontend can build against a mock server generated from the contract while backend implements the real logic.

39.2 Real Contract-First Workflow



39.3 Consumer-Driven Contract Testing (Pact)

Rather than the API provider unilaterally deciding what the contract is, each **consumer** (e.g., the React frontend, or another microservice) defines the exact expectations it depends on as a "contract" — the provider's CI pipeline runs these consumer contracts as automated tests, immediately catching any change that would break a real consumer, before it's ever deployed.

```
// Pact consumer test (written by the frontend/consuming team)
@Pact(consumer = "customer-portal")
public RequestResponsePact createPact(PactDslWithProvider builder) {
    return builder
        .given("account 1001 exists")
        .uponReceiving("a request for account 1001")
        .path("/api/v1/accounts/1001")
        .method("GET")
        .willRespondWith()
        .status(200)
        .body(newJsonBody(o -> { o.numberType("id", 1001); o.stringType("status", "ACTIVE"); }).build())
        .toPact();
}

// This generated "pact file" is then run against the ACTUAL provider's code in ITS CI pipeline -
// if the provider changes the response shape and breaks this expectation, their build fails immediately
```

Real scenario: "Before Consumer-Driven Contracts, a well-intentioned rename of a field in the Accounts Service API silently broke the mobile app in production for two days before anyone noticed. Now, a Pact contract test defined by the mobile team runs in the Accounts Service's own CI pipeline — the exact same rename now fails the build immediately, long before it ever reaches production."

40. Resilience Patterns for APIs (Recap: Circuit Breaker, Retry, Bulkhead, Timeout, Fallback)

These patterns are covered in full depth in the Microservices guide — this section is a focused recap specifically from the API design perspective: every API that calls another API (internal or third-party) should apply all five together, not just one in isolation.

Pattern	API-Specific Application
Timeout	Every outbound HTTP client call must have an explicit connect + read timeout — never rely on defaults
Retry	Only retry idempotent calls (GET, PUT, DELETE) or POST calls protected by an idempotency key
Circuit Breaker	Wraps the outbound call; opens fast once a downstream API's failure rate crosses a threshold
Bulkhead	Isolates the thread pool/connections used for one downstream API from others, preventing one slow dependency from starving unrelated calls
Fallback	Defines what to return when all else fails — cached/stale data, a default value, or a clear "temporarily unavailable" error to the client

```
@CircuitBreaker(name = "creditBureau", fallbackMethod = "fallbackScore")
@Retry(name = "creditBureau")
@Bulkhead(name = "creditBureau")
@TimeLimiter(name = "creditBureau")
public CompletableFuture<CreditScoreResponse> getCreditScore(String customerId) {
    return CompletableFuture.supplyAsync(() -> creditBureauClient.getScore(customerId));
}

public CompletableFuture<CreditScoreResponse> fallbackScore(String customerId, Throwable t) {
    return CompletableFuture.completedFuture(CreditScoreResponse.unavailable()); // graceful degradation
}
```

41. Event-Driven & Asynchronous API Communication

41.1 When a Synchronous API Response Isn't the Right Model

Some operations genuinely take too long for a client to wait on synchronously (a loan risk assessment taking 30+ seconds, a bulk file processing job) — the API should accept the request immediately and let the client check back later or be notified when done, rather than holding a connection open.

41.2 The 202 Accepted + Polling Pattern

```
POST /api/v1/loans/applications
→ 202 Accepted
{ "applicationId": "APP-9921", "status": "PROCESSING",
  "statusUrl": "/api/v1/loans/applications/APP-9921" }

# Client polls the status URL until it's done
GET /api/v1/loans/applications/APP-9921
→ 200 OK
{ "applicationId": "APP-9921", "status": "APPROVED" }
```

41.3 Webhook Pattern (Push Instead of Poll)

```
// Client registers a callback URL when submitting the request
POST /api/v1/loans/applications
{ "amount": 500000, "callbackUrl": "https://partner.com/webhooks/loan-status" }

// Our system POSTs to that URL once processing completes – no polling needed
POST https://partner.com/webhooks/loan-status
{ "applicationId": "APP-9921", "status": "APPROVED" }
```

41.4 Event-Driven Integration (Kafka) Behind an API

Internally, the API might simply validate the request and publish an event to Kafka (Microservices guide, Section 24) for asynchronous processing by other services — the API's synchronous contract with the client (202 Accepted) is deliberately decoupled from the internal asynchronous processing that follows.

42. SLA & SLO Considerations for APIs

Full conceptual detail is in the Observability & Monitoring guide (Section 24) — applied specifically to APIs: define and publish concrete targets per endpoint or per API tier (critical vs non-critical), and monitor real production SLIs against them continuously.

API Tier	Example SLO
Critical (e.g., balance check, payment)	99.95% availability, p99 latency < 500ms
Standard (e.g., transaction history)	99.9% availability, p99 latency < 1.5s
Batch/Reporting APIs	99% availability, latency less critical, completion time SLA instead (e.g., "within 10 minutes")

Real scenario: "Our public partner-facing API has a contractual SLA of 99.9% uptime with financial penalties for breach — internally, we set our own SLO at 99.95% specifically to give the team an early warning buffer before ever risking an actual SLA breach and its financial consequences."

43. Postman and Other API Development Tools

43.1 Postman — The Standard API Testing Tool

Postman lets developers manually construct and send API requests (any method, headers, body, auth), organize them into Collections, chain requests together with scripts, and run automated Collection-based test suites — the de facto standard tool for manually exploring and testing APIs during development.

43.2 Real Postman Features Used Day to Day

Feature	Real Use
Collections	Organize all of Accounts Service's endpoints into one shared, version-controlled collection
Environments	Switch base URLs/tokens instantly between Dev/QA/Staging/Prod without editing each request
Pre-request Scripts	Auto-fetch a fresh JWT token before every request in an environment
Tests (post-response scripts)	Assert status code, response shape, and field values automatically
Newman (CLI runner)	Run a full Postman Collection as part of a CI/CD pipeline step, not just manually
Mock Servers	Postman can serve mock responses from a collection, letting frontend teams build against a fake API before the real one exists

```
// Postman Test script example – runs automatically after each request
pm.test("Status code is 200", () => pm.response.to.have.status(200));
pm.test("Response has balance field", () => {
  const json = pm.response.json();
  pm.expect(json).to.have.property("balance");
});
```

43.3 Other Important API Tools

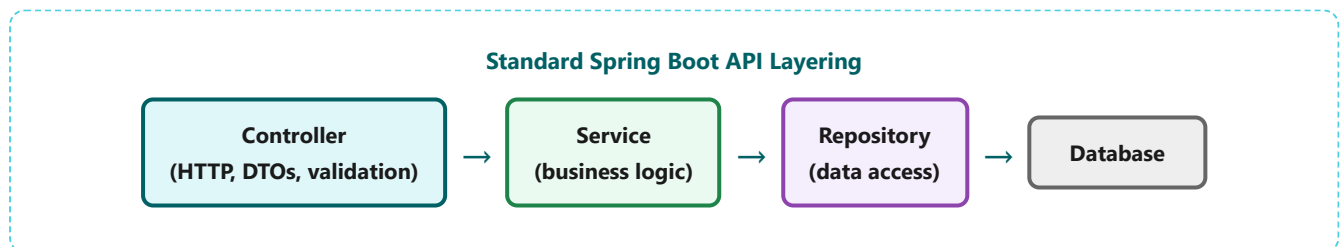
Tool	Purpose
curl	Command-line HTTP client — quick scriptable checks, used everywhere including CI pipelines
Insomnia	A popular lightweight Postman alternative
Swagger UI	Interactive documentation-driven testing directly from the OpenAPI spec (Section 16)
HTTPIe	A more human-friendly command-line HTTP client than raw curl
SoapUI	Testing tool specifically built for SOAP web services (and also supports REST)
grpcurl / BloomRPC	curl/Postman-equivalents specifically for testing gRPC services
k6 / JMeter / Gatling	Load/performance testing tools to verify an API meets its SLO under real traffic volume

44. How We Implement APIs in Our Organization (End-to-End Spring Boot)

44.1 The Full Implementation Checklist

1. **Design first:** agree on the OpenAPI spec/schema with consuming teams before writing implementation code (Section 39).
2. **Scaffold the controller layer:** resource-based URLs, correct HTTP methods, versioned base path (`/api/v1/...`).
3. **Define Request/Response DTOs** separately from JPA entities, with Bean Validation annotations.
4. **Implement the service layer** with business logic, calling repositories/other services as needed.
5. **Add a Global Exception Handler** producing a consistent error response shape.
6. **Wire up Swagger/OpenAPI annotations** so documentation is always accurate and live.
7. **Secure the endpoints** with Spring Security + JWT/OAuth2, role-based access per endpoint.
8. **Add resilience** for any outbound calls the API makes (timeout, retry, circuit breaker).
9. **Add observability:** structured logging with correlation IDs, Actuator health checks, Micrometer metrics.
10. **Write tests:** unit tests for the service layer, `@WebMvcTest` / `@SpringBootTest` for the controller layer, Pact contract tests for key consumers.
11. **Deploy behind the API Gateway**, registered with Eureka, with rate limiting and routing configured.
12. **Monitor in production** via Grafana/Splunk dashboards built specifically for this API's Golden Signals.

44.2 Real Layered Architecture (Spring Boot)



```
@RestController
@RequestMapping("/api/v1/accounts")
@RequiredArgsConstructor
public class AccountsController {
    private final AccountsService accountsService; // controller depends only on the service, never the
    repository directly

    @GetMapping("/{id}")
    public AccountDto getAccount(@PathVariable Long id) {
        return accountsService.findById(id); // controller stays thin – no business logic here
    }
}

@Service
@RequiredArgsConstructor
public class AccountsService {
    private final AccountRepository accountRepository;
    private final AccountMapper mapper;

    public AccountDto findById(Long id) {
        Account entity = accountRepository.findById(id)
            .orElseThrow(() -> new AccountNotFoundException(id)); // business rules and exceptions live here
        return mapper.toDto(entity);
    }
}
```

45. APIs and Databases — DTO vs Entity, Avoiding the N+1 Query Problem

45.1 Recap — Never Return Entities Directly (Section 9)

Always map `@Entity` classes to DTOs before returning them from an API — this is worth repeating specifically in the database context because JPA's lazy-loaded relationships can also cause serialization errors or accidentally expose internal fields if entities leak directly into a JSON response.

45.2 The REST/JPA N+1 Problem (Not Just a GraphQL Issue)

```
// Naive endpoint — looks innocent, hides an N+1 problem
@GetMapping
public List<AccountDto> getAllAccounts() {
    List<Account> accounts = accountRepository.findAll(); // 1 query
    return accounts.stream()
        .map(a -> new AccountDto(a.getId(), a.getBalance(), a.getCards().size())) // a.getCards() triggers a
        SEPARATE query PER account!
        .toList();
}

// Fixed — fetch accounts AND their cards in one query using a JOIN FETCH
@Query("SELECT a FROM Account a LEFT JOIN FETCH a.cards")
List<Account> findAllWithCards();
```

Interview tip: Always enable Hibernate's SQL logging (`spring.jpa.show-sql=true`) in development and actually look at the query count for list endpoints — the N+1 problem is easy to introduce accidentally and easy to miss until it causes real production slowness under load.

45.3 Database Design Considerations Driven by the API

- Add indexes on columns used in common API filter/sort parameters (Section 11) — an unindexed sort column on a large table is a common real-world slow-API root cause.
- Use database-level pagination (`LIMIT/OFFSET` or keyset pagination) — never fetch everything into memory and paginate in application code.
- Keep transaction boundaries tight around the service layer method, not spanning the entire HTTP request lifecycle.

46. How the Frontend (React) Consumes Our APIs

46.1 Real React API Call (Using a Typed Client)

```
// Auto-generated typed client from our OpenAPI spec (Section 16) – no hand-written fetch/axios boilerplate
import { AccountsApi } from './generated-api-client';

const accountsApi = new AccountsApi({ basePath: 'https://api.mybank.com', accessToken: getJwtToken });

function AccountDetails({ accountId }) {
  const [account, setAccount] = useState(null);
  useEffect(() => {
    accountsApi.getAccount(accountId)
      .then(response => setAccount(response.data))
      .catch(error => handleApiError(error)); // generic error handler using the standard envelope (Section 24)
  }, [accountId]);

  return account ? <div>Balance: {account.balance}</div> : <Spinner />;
}
```

46.2 Handling Loading, Error, and Empty States

Every real frontend API call needs to handle at least three states: loading (show a spinner), error (show a friendly message, log the error with its correlation ID), and success/empty (show data, or a clear "no data" message) — a surprisingly common source of production bugs is only handling the success case.

46.3 Token Refresh & 401 Handling

```
// Axios interceptor – automatically refresh an expired JWT and retry the original request once
axios.interceptors.response.use(
  response => response,
  async error => {
    if (error.response?.status === 401 && !error.config._retry) {
      error.config._retry = true;
      const newToken = await refreshAccessToken();
      error.config.headers.Authorization = `Bearer ${newToken}`;
      return axios(error.config); // retry the original request with the fresh token
    }
    return Promise.reject(error);
  }
);
```

46.4 CORS and Environment-Specific API URLs (Recap)

The React app is built once per environment (or reads a runtime `env-config.js`, per the Docker & Kubernetes guide Section 9.4) pointing to the correct API base URL per environment, and the backend's CORS configuration (Section 20) must explicitly allow that exact frontend origin.

47. Types of Applications That Consume APIs (Web, Mobile, Third-Party, IoT)

47.1 Consumer Types and Their Different Needs

Consumer Type	Real Example	Special Considerations
Web Frontend (SPA)	React customer portal	CORS, browser-based token storage, needs fast p95 latency for good UX
Mobile App (iOS/Android)	Bank mobile app	Unreliable/slow networks — needs aggressive timeouts, retry, offline caching, smaller payloads matter more
Server-to-Server (other microservices)	Loans Service calling Accounts Service	Service-to-service auth (client credentials), higher throughput expectations, internal network only
Third-Party Partners	A fintech partner integrating with our loan API	Strict versioning/backward compatibility, API keys/OAuth2, rate limiting, detailed public documentation
IoT/Embedded Devices	ATM machines, POS terminals reporting transactions	Very constrained bandwidth/compute, often binary protocols or minimal JSON, must handle intermittent connectivity gracefully
Batch/Scheduled Jobs	Nightly reconciliation job pulling transaction data	Bulk pagination, higher tolerance for latency, but strict data completeness/consistency requirements

47.2 Designing One API to Serve Multiple Consumer Types

Rather than building a separate API per consumer type, real organizations design one well-versioned core API and, where needs genuinely diverge (e.g., mobile needing smaller payloads), use a **Backend-for-Frontend (BFF)** layer per consumer type that adapts/aggregates the core API rather than duplicating business logic.

Backend-for-Frontend (BFF) Pattern

Mobile BFF (smaller payloads)

Web BFF (full data)

Partner BFF (strict versioning, API keys)

↓ all call the same underlying

Core Accounts/Cards/Loans Services

48. Third-Party API Integration & External API Calls

48.1 Real Examples of Third-Party APIs in a Banking System

Third-Party API	Purpose
Credit Bureau API	Fetch a customer's credit score during loan underwriting
Card Network API (Visa/Mastercard)	Authorize and settle card transactions
Payment Gateway API	Process online payments/transfers
SMS/Email Provider API (Twilio, SendGrid)	Send OTPs and notifications
KYC/Identity Verification API	Verify customer identity documents during onboarding

48.2 Why External API Calls Need Extra Care

Third-party APIs are outside your control — they can be slower, less reliable, change without much notice, or have usage-based costs. Every external call must be wrapped in the full resilience pattern set (Section 40): timeout, retry (only if idempotent), circuit breaker, and a sensible fallback.

48.3 Real Integration Pattern — Anti-Corruption Layer

Never let a third-party API's data shapes/quirks leak directly into your core domain model — wrap every external integration in a dedicated adapter/client class that translates the third party's format into your own clean internal DTOs, so a change in the third party's API only requires updating one adapter, not code scattered across your services.

```
@Component
public class CreditBureauAdapter {
    private final CreditBureauFeignClient externalClient;    // talks to the messy external API

    public CreditScore getCreditScore(String customerId) {
        ExternalBureauResponse raw = externalClient.fetchScore(customerId); // their weird format
        return new CreditScore(raw.getScoreValue(), mapRating(raw.getRatingCode())); // our clean internal
model
    }
}
```

48.4 Handling API Keys and Secrets for External Calls

Third-party API keys/secrets are stored in Azure Key Vault (via Variable Groups, per the CI/CD guide) — never hardcoded, never committed to source control, injected as environment variables at runtime, and rotated periodically per the vendor's security guidelines.

48.5 Monitoring External Dependencies Separately

Dashboard panels should distinguish "our error rate" from "third-party API error rate" — if the Credit Bureau API is down, that's a different incident category (and different escalation path, often to a vendor support contract) than a bug in our own Loans Service code.

Real scenario: "During a Card Network API outage, our Grafana dashboard's 'External Dependency Health' panel immediately showed 100% failure specifically for that one integration while our own service's internal error rate stayed near zero — letting us confidently tell customers 'card payments are delayed due to a third-party provider issue' instead of implying our own systems were broken."

49. Common API Issues, Problems & Solutions

Issue	Cause	Real Solution
API breaks existing clients after a "small" change	No backward compatibility discipline, no contract testing	Enforce additive-only changes, use API versioning, add Consumer-Driven Contract tests (Section 39)
CORS errors from the frontend	Backend doesn't explicitly allow the frontend's origin	Configure exact allowed origins in Spring Security/CORS config (Section 20)
Duplicate records created from double-clicks/retries	POST endpoint not idempotent	Implement Idempotency-Key pattern (Section 13)
Slow list endpoints under load	Missing pagination, N+1 query problem, missing DB indexes	Enforce pagination everywhere, use JOIN FETCH/DataLoader, add appropriate indexes (Sections 11, 45)
Sensitive internal details leaked in error responses	Raw exception messages/stack traces returned to clients	Global exception handler returns generic safe messages, logs full details server-side only (Section 14)
API documentation out of sync with actual behavior	Manually maintained docs instead of generated from code	Adopt springdoc-openapi/Swagger, generate docs directly from annotated code (Section 16)
Cascading failures when a downstream/third-party API is slow	No timeout/circuit breaker on outbound calls	Apply full resilience pattern set on every outbound call (Sections 28, 40)
Clients hardcoding URL structures break when routes change	No use of HATEOAS or clear documented URL contracts	Communicate URL contract changes clearly via versioning; consider HATEOAS for long-lived public APIs
Rate limit hit unexpectedly by a legitimate high-volume partner	Rate limits not tiered/negotiated per consumer	Offer tiered rate limits per API key/partner agreement, clear 429 responses with Retry-After headers
GraphQL query causes server overload	No query depth/complexity limiting	Enforce max depth, query cost analysis, persisted queries (Section 33)
Inconsistent field naming/response shape across services	No org-wide REST API standards enforced	Establish and enforce naming conventions and a standard response envelope (Section 24) via shared starter libraries/lint rules
Clients see intermittent 502/504 errors under load	Gateway/load balancer timeout shorter than the backend's actual processing time	Align timeout values across every hop (client, Gateway, service), and move genuinely slow operations to async (202 Accepted + polling, Section 39/41)
API works in Postman but fails from the React app	Almost always a CORS misconfiguration or a missing/expired auth token in the actual browser request	Compare request headers between Postman and the browser Network tab; fix allowed origins (Section 20) or token attachment logic
Two services return different date/number formats for the same data	No org-wide serialization standard (e.g., one uses epoch millis, another ISO-8601)	Standardize on ISO-8601 for dates and enforce it via a shared Jackson configuration starter across all services
A partner integration breaks every few months	No automated contract tests catching accidental breaking changes before release	Adopt Consumer-Driven Contract testing (Section 39) so breaking changes fail the CI build, not the partner's production system

50. Interview Questions & Answers (Basic → Advanced → Scenario Based)

Q1. What is an API and why do we need it?

An API is a defined contract letting one piece of software request data/services from another without knowing its internal implementation. It's needed to decouple systems, enable real-time integration (vs stale file exports), enforce security boundaries, and let one backend serve many different consumers (web, mobile, partners) consistently.

Q2. What's the difference between REST and SOAP?

SOAP is a strict, XML-only protocol with a formal WSDL contract and built-in enterprise standards (WS-Security); REST is a lighter architectural style, usually JSON, built around HTTP verbs and resources, with more flexible/optional contracts (OpenAPI) — REST is far more common today due to its simplicity and performance.

Q3. Explain the difference between PUT and PATCH.

PUT replaces the entire resource with the given representation and is idempotent. PATCH applies a partial update to specific fields only, and isn't guaranteed idempotent depending on implementation (e.g., "increment balance by 10" applied twice produces a different result each time).

Q4. Why shouldn't you return JPA entities directly from a REST controller?

It tightly couples your public API contract to your internal database schema, risks exposing sensitive/internal fields, and can cause serialization issues with lazy-loaded relationships. Always map entities to dedicated DTOs designed specifically for the API contract.

Q5. What is idempotency and how do you achieve it for a POST endpoint?

Idempotency means repeating an operation produces the same result as doing it once. POST isn't idempotent by default; you achieve it using an Idempotency-Key header — the server checks if that key was already processed and returns the original result instead of repeating the side effect (e.g., preventing duplicate payments).

Q6. What is the purpose of Swagger/OpenAPI, and how do you implement it in Spring Boot?

OpenAPI is the specification format describing an API's endpoints/schemas; Swagger is the tooling built around it (interactive UI, codegen). In Spring Boot, adding springdoc-openapi auto-generates the spec and an interactive Swagger UI directly from your controller annotations, keeping documentation always in sync with the actual code.

Q7. How would you handle a breaking API change without disrupting existing consumers?

Introduce a new API version (e.g., /api/v2/) with the breaking change, keep the old version running in parallel, communicate a deprecation timeline via Deprecation/Sunset headers and documentation, migrate consumers gradually, then retire the old version once no traffic remains.

Q8. What is the N+1 problem, and how do you fix it in a REST API backed by JPA?

It's when fetching a list triggers one query for the list plus one additional query per item for a related entity (e.g., fetching each account's cards separately) — N+1 total queries instead of a couple. Fixed using JOIN FETCH queries, entity graphs, or batch fetching to retrieve related data in far fewer queries.

Q9. Difference between JWT and OAuth2?

JWT is a token format (a signed, self-contained set of claims). OAuth2 is an authorization framework defining how tokens (often JWTs) are issued, scoped, and used across different grant types (Authorization Code, Client Credentials, etc.) — you can use JWTs without OAuth2 and OAuth2 with non-JWT opaque tokens.

Q10. What is CORS and why does it only affect browsers?

CORS (Cross-Origin Resource Sharing) is a browser-enforced security mechanism blocking web pages from calling APIs on different origins unless explicitly allowed via response headers. It's enforced by the browser itself, so non-browser clients (Postman, curl, mobile apps, server-to-server calls) are never affected by CORS.

Q11. What's the difference between GraphQL and REST?

REST exposes multiple fixed-shape endpoints per resource, risking over/under-fetching; GraphQL exposes one flexible endpoint where the client specifies exactly the fields it needs, aggregating multiple sources in one request — at the cost of harder HTTP caching and needing careful query complexity limiting.

Q12. What is the N+1 problem in GraphQL and how is it solved?

Resolving a list of parent objects and then a related field per item (e.g., cards per account) naively triggers one data-fetch call per item. DataLoader solves this by batching and deduplicating those calls within a single request tick into one bulk fetch instead of many individual ones.

Q13. When would you choose gRPC over REST?

For high-throughput, low-latency internal service-to-service calls where the performance/payload-size benefits of binary Protobuf over HTTP/2 clearly outweigh REST's simplicity — e.g., a fraud-scoring service called on every transaction, or use cases needing native bidirectional streaming.

Q14. What is Contract-First / API-First development, and why use it?

Designing and agreeing on the API contract (OpenAPI/Protobuf/GraphQL schema) before writing implementation code, so frontend, backend, and other consumer teams can work in parallel against the agreed contract (frontend against a generated mock), reducing integration surprises later.

Q15. What are Consumer-Driven Contracts and what problem do they solve?

Each API consumer defines the exact expectations it depends on as an automated "contract" (e.g., via Pact); the provider's CI pipeline runs these as tests, catching any accidental breaking change before it's deployed — solving the real problem of a well-intentioned change silently breaking a consumer in production.

Q16. How do you secure service-to-service API calls versus user-facing API calls?

User-facing calls typically use the Authorization Code (+PKCE) OAuth2 grant tied to an actual logged-in user's identity and roles. Service-to-service calls typically use the Client Credentials grant — a machine identity with its own scopes, no end user involved — validated the same way at the resource server via JWT/OAuth2.

Q17. How would you design an API to support a long-running operation like a loan risk assessment?

Return 202 Accepted immediately with a status URL the client can poll, or accept a callback/webhook URL to notify the client asynchronously when processing completes — avoiding holding an HTTP connection open for a slow operation.

Q18. How do you handle a slow or unreliable third-party API dependency?

Wrap every call in a timeout, retry (only if safe/idempotent), circuit breaker, and fallback; isolate it with an anti-corruption layer/adaptor so its quirks don't leak into your domain model; and monitor its health separately from your own service's health so incidents are correctly attributed.

Q19. What real API-related production issue have you resolved, and how?

Example: a list endpoint became progressively slower as data grew, traced to Hibernate's lazy-loading triggering an N+1 query pattern (visible after enabling SQL logging) — fixed with a JOIN FETCH query, reducing query count from hundreds to one and cutting p95 latency dramatically.

Q20. How do you ensure API documentation stays accurate over time?

Generate it directly from code using springdoc-openapi/Swagger annotations rather than maintaining a separate manual document — since it's generated from the same source as the implementation, it can't silently drift out of sync, and it's reviewed as part of normal code review on every pull request.

Q21. Why does an API work fine in Postman but fail with a CORS error from the React app?

Postman is not a browser, so it never enforces CORS — only real browsers apply the CORS check on JavaScript-initiated requests. A failure only in the browser almost always points to a missing/incorrect Access-Control-Allow-Origin configuration on the backend, not a problem with the API logic itself.

Q22. What status code should you return for a validation error, and what should the response body contain?

400 Bad Request, with a structured body listing each invalid field and a clear message (e.g., `{"errors": [{"field": "email", "message": "must be a valid email"}]}`) — generic, unstructured error text forces the frontend to guess which field to highlight.

Q23. How would you design an API endpoint to avoid becoming a bottleneck behind an API Gateway with a strict timeout?

Ensure the endpoint's own processing time is comfortably under the Gateway's configured timeout by optimizing slow queries/dependencies, and for genuinely long operations, switch to an asynchronous pattern (202 Accepted + status polling or a callback/webhook) instead of holding the connection open.

Q24. What's the risk of using different date/number formats across microservices, and how do you prevent it?

Inconsistent formats force every consumer to write custom, error-prone parsing per service, and subtle timezone/precision bugs slip through easily. Prevent it with an organization-wide serialization standard (ISO-8601 dates, consistent numeric types) enforced via a shared Jackson configuration library used by every service.

Q25. How do you design an API response envelope so all services look consistent to consumers?

Define one shared response wrapper (e.g., `{"data": ..., "error": null, "meta": {"traceId": ...}}`) used by every service via a common base library, so a consumer integrating with 10 different internal APIs sees exactly the same top-level shape every time, regardless of which team built the service.

51. Final Summary & Cheat Sheet

Before APIs: tightly coupled direct database access, stale nightly file exports, fragile point-to-point integrations.

REST fundamentals: resource-based URLs, correct HTTP verbs/status codes, DTOs (never expose entities), consistent versioning, pagination, robust error handling, and validation at the boundary.

Swagger/OpenAPI: the backbone of living, always-accurate documentation, interactive testing, and contract-first collaboration across teams.

Beyond REST: GraphQL for flexible, aggregated data-fetching needs (with careful N+1 and query-complexity handling); gRPC for high-throughput internal streaming needs; SOAP still alive in legacy enterprise integrations.

Production readiness: JWT/OAuth2 security, CORS correctly configured, rate limiting, resilience patterns (timeout/retry/circuit breaker/bulkhead/fallback) on every outbound call, correlation IDs for traceability, and full observability tied to Golden Signals.

The golden rule: design the contract first, keep it backward compatible, document it from code (not separately), and treat every downstream/third-party call as something that WILL eventually fail — build resilience in from day one, not after the first outage.

51.1 Quick Cheat Sheet

Term	One-Line Meaning
DTO	Object shape exposed via API, decoupled from internal database entities
Idempotency Key	Client-generated key preventing duplicate side effects on retried POST calls
HATEOAS	API responses include links describing available next actions
OpenAPI/Swagger	Spec + tooling for generating live, accurate API documentation
Correlation ID	Unique ID tying together all logs/calls for one request across services
Circuit Breaker	Fails fast once a downstream dependency's failure rate crosses a threshold
N+1 Problem	One query becomes many due to naive per-item related-data fetching (REST/JPA or GraphQL)
DataLoader	Batches/deduplicates GraphQL data-fetching calls to avoid N+1
JWT	Signed, self-contained token carrying identity/role claims
OAuth2	Framework defining how tokens are issued/used (Authorization Code, Client Credentials, etc.)
CORS	Browser-enforced rule controlling cross-origin API calls from web pages
Consumer-Driven Contract	Consumer-defined expectations tested automatically in the provider's CI pipeline
gRPC	Binary, HTTP/2-based, strongly-typed RPC framework — fast, streaming-capable
GraphQL	Single flexible endpoint where clients specify exactly the fields they need
Anti-Corruption Layer	Adapter isolating a messy third-party API's shape from your clean internal domain model

Use this guide alongside the Git/GitHub/GitLab/Bitbucket guide, the CI/CD guide, the Docker & Kubernetes guide, the Microservices guide, and the Observability & Monitoring guide — together they cover the complete real-world Java full-stack developer journey from writing and versioning code, to designing and securing production-grade APIs, to deploying, orchestrating, and observing them reliably at scale.